

03-30 Лекция: Автономные AI-агенты, OpenCore и Канбан

[Дата и время] : 2026-03-30 12:52:39

[Место: [Вставить место]] : [Вставить место]

[Преподаватель: [Вставить имя]] : [Вставить имя]

Краткое содержание

Лекция посвящена автономной работе AI-агентов на платформе OpenCore и методам организации их взаимодействия. Рассматриваются два основных пути автономного запуска задач: Cron для выполнения по строгому расписанию и Heartbeat для более гибких, но ресурсоемких запусков на основе текстовых инструкций. Обсуждается проблема взаимодействия множества специализированных агентов и предлагается решение на основе Канбан-доски, позволяющей агентам координировать свою работу без прямой коммуникации, что экономит ресурсы. Также разъясняется архитектура работы агента, включая его рабочее пространство (workspace), взаимодействие с языковой моделью (LLM) и каналы коммуникации, такие как Telegram. Подчеркивается, что рабочее пространство агента (папка с данными) является ключевым элементом, который можно переносить между различными платформами, такими как OpenCode или Codecks.

Тезисы

1. Автономная работа агентов

- Пути автономной работы
 - Существует два отдельных пути для автономной работы агентов: Cron и Heartbeat.
- Cron
 - Аналог службы cron в Linux, предназначенной для запуска задач по четкому расписанию.
 - Позволяет задать запуск конкретной задачи в определенный день, час и минуту.

- Пример: каждый понедельник в 9:00 утра запускать задачу по сбору новостей с Reddit и их отправке пользователю.
- Задача запускается автоматически, агент ее не видит. В задаче указывается, какого субагента запустить и с каким промптом.
- Это фоновая служба, которая срабатывает при совпадении заданных временных параметров.
- Такой подход ограничен и работает по строгому расписанию.
- **Heartbeat**
 - Второй путь автономной работы в OpenCore.
 - Heartbeat представляет собой текстовый файл (похожий на Markdown), в котором человеческим языком описано, что нужно делать.
 - В отличие от Cron, Heartbeat запускает всего агента целиком, предоставляя ему задачу и используя весь его багаж контекста, включая память (soul) и инструменты.
 - Запуск происходит не по строгому расписанию; агент может самоорганизоваться и запускаться раз в час, раз в два часа или с другой периодичностью.
 - Этот метод более гибкий, так как агент обладает полным контекстом (кто вы, что вы, как вас зовут), но требует осторожности, так как частые запуски могут быстро израсходовать бюджет на токены.

2. Организация взаимодействия агентов

- **Проблема коммуникации**
 - С ростом количества специализированных агентов (для здоровья, работы, поиска перелетов и т.д.) возникает проблема их эффективного взаимодействия.
 - Прямая коммуникация между агентами, когда один вызывает другого, является дорогой, так как вовлекает весь контекст и расходует много токенов.
 - Пример дорогой коммуникации: один агент спрашивает другого о билетах, тот, в свою очередь, уточняет детали у пользователя, и каждый такой обмен стоит денег.
- **Канбан-доска как решение**
 - Для организации эффективной работы нескольких агентов без дорогостоящей прямой коммуникации предлагается использовать методологию Канбан, придуманную японцами.

- Канбан-доска представляет собой общий файл (например, в Google Диске), к которому имеют доступ все агенты.
 - Доска имеет колонки, такие как “Backlog” (что нужно сделать в принципе), “To Do” (к исполнению), “In Progress” (в работе) и “Done” (выполнено).
 - Агенты получают задание периодически проверять доску.
- **Механизм работы с Канбан-доской**
 - Любой агент может добавить задачу в колонку “To Do”. Например, агент, следящий за здоровьем, может поставить задачу агенту-“билетеру” найти поездку к морю.
 - Агент, ответственный за соответствующую специализацию (например, поиск билетов), видит новую задачу, перемещает ее в “In Progress” и начинает над ней работать.
 - Это позволяет всем агентам видеть статус задач и исключает необходимость прямой коммуникации для постановки задач.
 - В системе может существовать список агентов и их специализаций, что помогает адресовать задачи правильно.
 - Система очень гибкая: можно добавлять колонки, устанавливать правила (например, удалять задачу из “Done” через неделю). Этот подход решает вопрос организации работы множества агентов.

3. Архитектура и среда работы агента

- **Локальная работа и Workspace**
 - Агенты OpenCore работают локально в выделенной папке на сервере (например, VPS). Эта папка называется “Workspace”.
 - Workspace содержит в виде файлов всю информацию об агенте: его память (soul), историю переписки, настройки и другие данные.
 - Агент запускается, читает всю информацию из этой папки и выполняет свои задачи. Результаты работы также сохраняются локально.
- **Взаимодействие с LLM и пользователем**
 - Программа OpenCore управляет взаимодействием. Она отправляет данные из Workspace вместе с запросом пользователя в языковую модель (LLM), такую как GPT или Gemini.
 - LLM генерирует ответ на основе полученного контекста, который OpenCore затем может переслать пользователю через канал общения, например, Telegram.

- Telegram (или Discord, Slack) является лишь одним из каналов общения с агентом.
- **Видимый и невидимый слои работы**
 - Существует видимый слой взаимодействия, когда пользователь напрямую общается с агентом (например, пишет “Привет”).
 - Существует и невидимый слой, когда агент запускается автономно (через Cron или Heartbeat) без участия пользователя для выполнения фоновых задач, например, сбора данных из Google Диска. Агент может решить ответить пользователю по результатам такой задачи.
- **Переносимость агента и Codecks**
 - Ключевым элементом архитектуры является папка Workspace. Её можно синхронизировать с GitHub.
 - Эту папку можно импортировать в другие среды, например, в Codecks. Codecks — это отдельное приложение со своим интерфейсом, которое может загрузить в себя данные из папки и работать с тем же агентом.
 - Это позволяет “пересаживать” агента между разными платформами (OpenCore, Codecks, Gemini in Antegravity), сохраняя его знания и способности. Сравнивается с флешкой из фильма “Терминатор”, которую можно вставить в другое устройство.
- **Интеграция и триггеры**
 - Для интеграции с внешними сервисами (например, соцсетями) и получения входящих данных можно использовать разные подходы.
 - Первый вариант — периодический опрос (polling), когда агент раз в N часов проверяет наличие новых данных (например, комментариев) через API.
 - Второй, более эффективный вариант — вебхуки (webhooks). Сервис сам отправляет уведомление агенту при наступлении события (например, новый комментарий). Telegram-бот работает по такому же принципу.

Вопросы

- [Вставить вопрос/непонимание]

Следующие шаги

- [] 1. Изучить возможности Cron и Heartbeat для настройки автономных задач для своих агентов.
- [] 2. Организовать взаимодействие нескольких агентов с помощью Канбан-доски (например, в Google Диске), настроив колонки и правила.
- [] 3. Для интеграции с внешними сервисами определить подход (опрос или вебхуки) под конкретную задачу и при необходимости проконсультироваться с OpenCore.
- [] 4. Попрактиковаться в переносе папки Workspace агента между различными средами (например, с локального сервера на Codecks через GitHub), чтобы понять принцип переносимости.